

DEEP RESEARCH / PRODUCT STRATEGY

The World-Class SMTP Framework

A research agenda for Simple Java Mail 10.0.0

PROTOCOL	OPERATIONS	INTEGRITY	TRUTH
Modern ESMTP	Bounded execution	Finalized bytes	Retry-safe outcomes

A cross-ecosystem study of .NET, Java, Node.js, PHP, Rust, Go, and Python libraries - translated into measurable release gates for a defensible robustness claim.

Contents

The report separates the competitive landscape, the Simple Java Mail gap analysis, the 10.0.0 release gates, and the evidence required for a public claim.

Executive verdict	3
1. Scope and method	3
2. What the competitive landscape actually says	4
3. Simple Java Mail's current strategic position	6
4. The 10.0.0 release gates	7
5. Priority map: what belongs in 10.0.0	10
6. What not to put into this claim	11
7. The public claim ladder	11
8. A proposed product thesis for 10.0.0	12
9. Standards baseline	13
10. Decision checklist	14
Conclusion	14
Research references	15

Executive verdict

Executive verdict. Simple Java Mail is already one of the broadest high-level outbound-mail frameworks, especially when composition, cryptography, proxying, connection reuse, exact finalized-EML handling, and a pleasant Java API are considered together. It is not yet the strongest SMTP protocol engine in the surveyed field. MailKit/MimeKit currently sets that bar; Nodemailer sets much of the operational-ergonomics bar; Symphony Mailer demonstrates framework-level failover and durable queue integration; Vert.x demonstrates what native asynchronous Java SMTP can look like. The credible route to the top is not another collection of builder conveniences. It is a truthful, protocol-complete submission contract that can answer, for every recipient and every failure boundary: **what happened, what may have happened, and whether retrying is safe.**

The short answer

To make a serious claim to being the most robust SMTP sending framework, Simple Java Mail must combine four bodies of strength:

1. **MailKit's protocol completeness:** negotiated PIPELINING, CHUNKING/BINARYMIME, SMTPUTF8, complete DSN, REQUIRETLS, strong SASL/TLS controls, cancellation, and protocol-level diagnostics.
2. **Nodemailer's operational discipline:** bounded pools and queues, backpressure, rate control, full per-recipient results, proxies, test transports, and useful timing metadata.
3. **Symphony's framework ergonomics:** transport failover, durable queue integration, terminal events, rich testing support, and high-level DKIM/S/MIME facilities.
4. **Simple Java Mail's own differentiators:** a cohesive Java API, dynamic OAuth credentials, proxy and cluster support, rehearsal, modern MIME security, and preservation of finalized wire content where rebuilding would invalidate signatures or alter bytes.

Then it must add the piece no surveyed competitor gets fully right: **typed transaction truthfulness.** That means per-recipient SMTP replies and enhanced status codes, an explicit indeterminate outcome after ambiguous disconnects, a derived retry-safety classification, real cancellation/deadline semantics, and public failure-injection evidence.

The resulting defensible claim is narrower and stronger than “best mailing library in the world”:

The most robust open-source framework for submitting email over SMTP, with complete recipient-level outcomes and explicit duplicate-risk semantics.

That sentence can be tested. “Best” without a published scorecard cannot.

1. Scope and method

This study compares libraries that construct and/or submit Internet messages to an SMTP server. It deliberately excludes commercial delivery APIs such as SendGrid and Mailchimp, inbound POP3/IMAP clients, and full mail-transfer agents that operate recipient-domain queues. The primary comparison set is:

- Simple Java Mail 9.3.2 plus the unreleased 10.0.0 milestone and local 10.0.0 branch.
- MailKit + MimeKit for .NET; Microsoft's `System.Net.Mail.SmtpClient` is retained only as a baseline because Microsoft recommends MailKit or another modern library for new development.

- Nodemailer for Node.js.
- Symfony Mailer + Mime for PHP.
- Vert.x Mail Client for Java.
- lettre for Rust, go-mail for Go, Python email/smtplib plus aiosmtplib, and PHPMailer.

The method favors official documentation, live source, RFCs, and active issue/release records. “Supported” means a documented first-class API or behavior confirmed in source. A missing item is reported as “not found as a first-class feature,” not as proof that a caller cannot reach it through raw protocol properties or custom extensions.

Five dimensions drive the comparison:

- **Protocol correctness and reach:** SMTP state, extension negotiation, internationalization, DSN, modern TLS/authentication, PIPELINING, and CHUNKING.
- **Message integrity:** MIME construction, DKIM, S/MIME, OpenPGP/MIME, and exact finalized bytes.
- **Outcome truth:** recipient-level replies, enhanced status codes, partial acceptance, the SMTP commit point, and retry ambiguity.
- **Operational robustness:** asynchronous execution, cancellation, deadlines, pools, bounded queues, backpressure, rate control, failover, and observability.
- **Proof:** interoperability, fault injection, benchmarks, maintenance, and reproducible conformance evidence.

The live [10.0.0 milestone](#) contained 19 issues at the research snapshot: 14 closed and 5 open. The open work included the protocol-oriented [PIPELINING/CHUNKING issue #699](#), plus packaging, Spring Boot, diagnostics, and Java 11 API modernization. Those are valuable, but only #699 directly changes the SMTP-engine ceiling.

2. What the competitive landscape actually says

There is no universal incumbent. Leadership is split by axis.

Axis	Current benchmark	Why it matters
Modern ESMTP engine	MailKit	Broad negotiated extension set, cancellable sync/async operations, strong TLS/SASL/proxy surface, and protocol logging.
Operational SMTP ergonomics	Nodemailer	Bounded connection pool, queue/backpressure hooks, rate limiting, full DSN, partial-recipient reports, timings, and test transports.
High-level framework integration	Symfony Mailer	Failover and round-robin transports, durable Messenger queues, events, testing assertions, and DKIM/S/MIME facilities.
Native asynchronous Java transport	Vert.x Mail Client	Event-loop-native API, pooled connections, queued acquisition, dynamic credentials, and negotiated PIPELINING.
MIME and cryptographic breadth	MimeKit and Simple Java Mail	Both cover rich MIME and several message-security mechanisms; SJM's finalized-EML path is a meaningful integrity differentiator.
Transaction truth and retry safety	No complete leader	Libraries expose pieces, but none combines every recipient reply, enhanced code, protocol phase, indeterminate completion, and derived retry disposition.

2.1 MailKit + MimeKit: the protocol benchmark

Microsoft's current documentation says `System.Net.Mail.SmtpClient` is not recommended for new development because it does not support many modern protocols, and directs users toward MailKit or another library. The meaningful .NET comparator is therefore MailKit/MimeKit, not the standard client. See [Microsoft's SmtpClient guidance](#) and [.NET compatibility note DE0005](#).

MailKit 4.17 exposes a formidable capability set: SIZE, DSN, enhanced-status-code discovery, AUTH, 8BITMIME, PIPELINING, BINARYMIME, CHUNKING, STARTTLS, SMTPUTF8, and REQUIRETLS. Its send APIs have synchronous and asynchronous variants, cancellation tokens, progress reporting, reusable connections, socket/stream injection, TLS policy hooks, proxies, broad SASL mechanisms, and a redacting protocol logger. The official surfaces are captured by its [SMTP capability enum](#), [SmtplibClient API](#), [send API](#), and [ProtocolLogger](#).

MimeKit adds rich MIME, S/MIME, OpenPGP/MIME, DKIM, and ARC. This pair is the bar SJM must meet before claiming superior SMTP breadth.

Its openings are important, however. A client instance must be serialized and is not a managed concurrent pool. Retry, rate, and queue policy remain caller concerns. The default recipient-rejection behavior aborts before DATA when any recipient is rejected; subclasses can change that behavior, but a full typed partial-delivery receipt is not the default. Three-digit SMTP status and raw text are exposed, but RFC 3463 enhanced x.y.z status is not modeled as a first-class type. MimeKit promises high-fidelity parsing and serialization, but no general byte-identical parse/reserialize contract was found. These are credible areas for SJM to lead rather than imitate.

2.2 Nodemailer: the operational benchmark

Nodemailer combines practical protocol coverage with excellent operations. Its [SMTP transport](#), [pooled transport](#), and response model expose accepted and rejected recipients, rejection errors, the final response, envelope and message timing, message size, and EHLO metadata. Pools offer connection/message limits, queuing, idle notifications, rate limits, and a requeue bound. Proxy and OAuth flows are first class.

Its [DSN API](#) is notably fuller than SJM's released DSN surface: RET, ENVID, per-recipient NOTIFY, and ORCPT are all modeled. [REQUIRETLS](#), DKIM, raw input, plugins, and a [stream transport](#) for deterministic tests further raise the bar.

Nodemailer's gaps define opportunity. It does not implement CHUNKING/BDAT, lacks a documented per-send cancellation contract, and does not provide built-in S/MIME or OpenPGP/MIME. Its raw SMTP path normalizes bare LF, so "raw" is not an unconditional exact-wire guarantee. Pool requeueing is operational recovery, not a protocol-aware idempotency system; a lost final acceptance reply can still create duplicate-risk ambiguity.

2.3 Symphony Mailer: the framework benchmark

Symphony's [Mailer documentation](#) demonstrates what a framework-level mail subsystem can feel like: failover and round-robin transport composition, rate limiting, TLS policy, persistent SMTP sessions, terminal events, debug output, profiler integration, test assertions, DKIM signing, and S/MIME signing/encryption. [Messenger](#) adds durable asynchronous queues and retry policy outside the SMTP transaction.

Its SMTP core is less ambitious. The current transport performs MAIL, RCPT, and DATA sequentially; no PIPELINING, CHUNKING, or typed DSN surface was found. A rejected recipient aborts before DATA, and `SentMessage` is not a recipient-level protocol receipt. Messenger provides durable job execution, not cancellable nonblocking SMTP. Failover also needs careful interpretation: trying another transport after an ambiguous post-DATA disconnect can duplicate mail unless the transport reports the commit uncertainty.

The lesson for SJM is not to absorb an application queue. It is to offer a clean terminal result and retry disposition that any queue, scheduler, or framework can use safely.

2.4 Vert.x Mail Client: the Java warning shot

The current [Vert.x Mail Client documentation](#) describes a fully asynchronous SMTP client with a connection pool, queued acquisition when the pool is busy, dynamic credential suppliers, message limits per connection, optional recipient-error continuation, and negotiated PIPELINING. Its [MailClient API](#) is native to the Vert.x execution model.

Vert.x does not match SJM's documented composition, message-security, finalized-content, and high-level result ambitions. It nevertheless makes a crucial point: a Java SMTP framework claiming world leadership cannot treat asynchronous execution as merely "run the blocking send on an unbounded executor." Native protocol progress, cancellation, bounded demand, and pressure propagation are part of robustness.

2.5 The typed modern challengers

Rust's [lettre](#) combines a typed API, secure TLS defaults, sync and async transports, connection reuse, SMTPUTF8/8BITMIME checks, raw submission, DKIM, and useful test/file transports. It lacks typed DSN, PIPELINING, CHUNKING, complete recipient receipts, and a truly bounded active pool. Its advertised pool maximum currently caps parked idle connections rather than all active concurrency.

[go-mail](#) is broader than many Java developers would expect: SMTPUTF8/8BITMIME, typed DSN RET/NOTIFY, extensive authentication, DKIM, S/MIME signing, EML import/export, middleware, reusable connections, and enhanced-status-aware errors. Its DSN is incomplete, recipient rejection prevents DATA, one connection serializes concurrent sends, and the current context path does not propagate cancellation through the entire SMTP exchange.

Python's [smtplib](#) remains a valuable semantic reference because `sendmail` returns every refused recipient when at least one recipient was accepted. The [email](#) package is a separate, rich MIME system. [aiosmtplib](#) adds coroutine-based SMTP. This combination is not a cohesive high-level framework: no managed pool, retry policy, signing system, lifecycle observer, or full typed DSN model is built in.

[PHPMailer](#) is an adoption and compatibility benchmark. It supports common MIME patterns, SMTPUTF8, TLS, XOAUTH2, multi-host fallback, connection persistence, partial delivery callbacks, DSN NOTIFY, DKIM, and S/MIME signing. It lacks modern async/cancellation, PIPELINING/CHUNKING, complete DSN, bounded pooling, and a structured submission receipt.

3. Simple Java Mail's current strategic position

Simple Java Mail already has a rare horizontal breadth:

- Structured message construction and conversion, attachments, embedded resources, calendar content, DKIM, S/MIME, and the 10.0.0 OpenPGP/MIME work.
- TLS/STARTTLS/SMTPS strategies, server identity checking, custom trust configuration, authentication, refreshable OAuth token providers, and authenticated proxy support.
- Connection pooling and clustering, lifecycle control, asynchronous APIs, and provider-neutral transport work.
- Send-time rehearsal, explicit sync/async direction, safe configuration diagnostics, Spring integration, and a CLI daemon direction.
- Finalized EML submission designed to preserve the exact message content and envelope through the normal transport path-essential for signatures and deterministic retry.
- A result vocabulary that already recognizes accepted, partially accepted, rejected, and unknown outcomes.

The last two points are strategically excellent. They move SJM toward the one property the market still lacks: honest protocol outcomes attached to immutable transmitted content.

But the local unreleased 10.0.0 model currently stops short of that promise. `MailSubmissionReceipt` groups accepted, valid-but-unsent, and invalid recipients and retains one overall SMTP response. `SmtplibServerResponse` stores a numeric reply and raw response text. It does not model every recipient's primary reply, enhanced `x.y.z` code, command phase, or raw reply. `MailSubmissionStatus.UNKNOWN` warns about duplicate risk, but that risk is not yet a derived retry contract. Asynchronous work uses a blocking transport behind executor dispatch, and the default executor has an unbounded queue; cancellation of the returned future does not establish cancellation of the underlying SMTP exchange.

Protocol breadth has similar gaps:

- [Issue #699](#) correctly targets PIPELINING and CHUNKING through a pluggable transport because the stock Angus transport remains lockstep.
- DSN was released in 9.0.0 via [issue #574](#), but the first-class SJM surface covers NOTIFY and RET, not the full RFC 3461 ENVID and ORCPT model.

- No first-class REQUIRETLS control or structured SMTPUTF8 capability requirement was found in the audited 10.0.0 API.
- 8BITMIME can be reached via provider properties, but protected or exact content must not be silently rewritten merely to fit server capabilities.
- Connection/read/write timeout settings exist, but an end-to-end deadline and protocol-phase cancellation contract are different guarantees.

This is not a negative assessment. It identifies where 10.0.0 can turn several good pieces into a coherent architecture before public API shapes freeze.

4. The 10.0.0 release gates

The following are the minimum gates for a “grand” release whose north star is world-class SMTP robustness. They are ordered by architectural leverage, not implementation convenience.

Gate 1 - Freeze a truthful submission-result model

The result model is more important than any single extension because every transport, observer, queue, retry policy, and framework integration will depend on it.

For each submission, retain:

- Requested, preparation-complete, transaction-start, and terminal timestamps.
- Endpoint and transport identity, advertised and post-TLS capabilities, selected authentication mechanism, negotiated TLS version/cipher, and whether identity verification succeeded-without secrets.
- A protocol timeline or at least the terminal phase: connect, greeting, EHLO, STARTTLS, post-TLS EHLO, AUTH, MAIL, each RCPT, DATA/BDAT, final acceptance reply, or cleanup.
- For every recipient: the original address, normalized envelope address, whether RCPT was attempted, the three-digit SMTP reply, parsed enhanced status, reply text, and one of ACCEPTED_FOR_TRANSACTION, TEMPORARILY_REJECTED, PERMANENTLY_REJECTED, NOT_ATTEMPTED, or OUTCOME_UNKNOWN.
- Final message state: definitely not submitted, partly accepted, definitely accepted by this SMTP server, or indeterminate.
- Original provider exception and a sanitized diagnostic record.
- A derived retry disposition: SAFE_TO_RETRY_ALL, SAFE_TO_RETRY_UNACCEPTED_ONLY, DO_NOT_RETRY, DUPLICATE_RISK, or CALLER_POLICY_REQUIRED.

“Accepted” must always mean the SMTP peer returned the final success reply. It must never imply inbox placement or final delivery. DSN requests do not change that truth.

The critical boundary is the disconnect after the client has transmitted the final DATA terminator or BDAT LAST, but before it receives the final reply. The server may have accepted the message. SMTP has no standardized idempotency key, and neither Message-ID nor DSN ENVID is a deduplication guarantee. The only honest terminal state is OUTCOME_UNKNOWN, and automatic retry must not hide the duplicate risk.

Gate 2 - Make modern ESMTP capability requirements first class

Implement and expose negotiated behavior-not raw provider-property folklore-for:

- PIPELINING per [RFC 2920](#), correlating every response by command order and handling mixed recipient results correctly.
- 8BITMIME per [RFC 6152](#), based on the finalized body actually transmitted.

- SMTPUTF8 per [RFC 6531](#) and RFC 6532, based on both envelope and headers. Fail clearly when required but unavailable; do not invent a lossy downgrade for a non-ASCII local part.
- SIZE per [RFC 1870](#), computed from finalized transmitted octets and compared with any advertised maximum before submission.
- Enhanced status codes per [RFC 2034](#), RFC 3463, and the IANA registry.
- Complete DSN per [RFC 3461](#): RET, ENVID, per-recipient NOTIFY, and ORCPT, with internationalized forms from RFC 6533.
- Per-message REQUIRETLS per [RFC 8689](#). Explain that this asks the submission chain to propagate a TLS requirement; it is not end-to-end encryption.

Expose capability snapshots and explicit requirement policies such as REQUIRE, USE_IF_AVAILABLE, and DISABLE where those semantics make sense. A library should never silently rewrite finalized or cryptographically protected content to compensate for a missing capability.

Gate 3 - Complete #699 without corrupting failure semantics

PIPELINING should ship first and be independently testable. It materially reduces round trips, especially for multiple recipients, but it also makes reply correlation and mixed RCPT outcomes easier to get wrong. The standard requires consuming every response and matching by command order, not by text.

CHUNKING/BDAT and BINARYMIME are valuable, especially for large or binary content, but they enlarge the failure-state space. The implementation must:

- Use BDAT only when CHUNKING is advertised.
- Count exact octets, never mix DATA and BDAT in one transaction, and stop/reset correctly after a chunk rejection.
- Require CHUNKING for BINARYMIME and use BODY=BINARYMIME; never send BINARYMIME through DATA.
- Preserve a precise commit boundary for BDAT LAST and return indeterminate when the final reply is lost.
- Fall back to DATA/7-bit-safe content only when doing so preserves the chosen content-integrity policy.

The feature is complete only when latency benchmarks and failure-injection tests show the negotiated path and every fallback path.

Gate 4 - Give cancellation and deadlines protocol meaning

`CompletableFuture.cancel()` changing a Java object is not the same as cancelling SMTP I/O. Define:

- A total submission deadline plus optional phase deadlines.
- Cancellation propagation through pool acquisition, DNS/connect, TLS, AUTH, command reads, message streaming, and cleanup.
- The outcome returned when cancellation happens before MAIL, during RCPT, during message bytes, or after the commit boundary.
- Connection disposal rules after cancellation or protocol ambiguity.
- Progress for large messages, so callers can distinguish queue wait from network transfer.

A cancellation request after the commit boundary cannot turn the result into “not sent.” It must return accepted if the final success was observed, or indeterminate if it was not.

Gate 5 - Bound every queue and make pressure visible

Robustness under overload requires finite resource ownership. Replace implicit unbounded submission demand with explicit policies:

- Maximum queued submissions, maximum concurrent transactions, and per-route or per-server connection limits.
- REJECT, BLOCK_UNTIL_DEADLINE, and caller-supplied overflow policies.
- Observable queue wait, pool wait, active connections, connection age, messages per connection, discard reasons, and saturation.
- Idle health checks, maximum connection age, and automatic disposal after protocol corruption, TLS/auth failure, timeout, cancellation, or indeterminate completion.
- Optional rate control as a 10.x operational module if it cannot fit the 10.0 core.

This is where Nodemailer and Vert.x provide better defaults today. SJM can surpass them by attaching backpressure to the typed outcome and deadline system.

Gate 6 - Secure submission defaults that are hard to misuse

Submission should make secure choices explicit and diagnosable:

- Implicit TLS on 465 and STARTTLS on 587; required TLS by default for authenticated submission.
- TLS 1.2 minimum and TLS 1.3 preferred, following current BCP 195 guidance.
- Certificate path and reference-identity validation against SAN; no silent trust-all shortcut in production-oriented defaults.
- Discard pre-TLS EHLO capabilities and issue EHLO again after STARTTLS.
- Do not send passwords or bearer tokens before verified TLS unless the caller makes an explicit unsafe compatibility choice.
- Pluggable SASL with standard OAUTHBEARER and refreshable token suppliers; label LOGIN/XOAUTH2 as interoperability mechanisms rather than modern standards leadership.
- Structured, redacted diagnostics that show policy and negotiation without credentials, message bodies, private keys, or bearer tokens.

Gate 7 - Prove finalized-message integrity

SJM's finalized-EML direction should become a flagship guarantee:

- The exact finalized network-form content used for SIZE is the content signed, encrypted, rehearsed, observed, submitted, and retrieved.
- DKIM is applied before SMTP transparency encoding; no mutable headers or transfer encoding can be introduced afterward.
- S/MIME targets current S/MIME 4.0 algorithms and certificate validation. OpenPGP/MIME uses a maintained RFC 9580-capable crypto implementation while preserving RFC 3156 MIME rules.
- Envelope data remains separate from headers, Bcc is handled correctly, and raw EML submission requires an explicit envelope.
- Tests verify byte identity before DATA transparency or BDAT framing and strict verification by independent tooling.

This is a real differentiator from libraries whose “raw” path reparses, reserializes, or normalizes content.

Gate 8 - Ship safe diagnostics and a capability probe

Developers need to diagnose “works locally, fails in production” without enabling a secret-leaking wire dump. Provide:

- A redacting protocol transcript with authentication payloads and message content excluded by default.
- Structured phase timings and classified failures.

- A read-only SMTP probe that reports greeting, post-TLS EHLO capabilities, SIZE maximum, authentication mechanisms, TLS version/cipher/certificate identities, and policy mismatches.
- Stable observation hooks with optional Micrometer and OpenTelemetry adapters, not mandatory core dependencies.
- Correlation identifiers that do not imply delivery or deduplication.

MailKit's protocol logger and telemetry are the benchmark. SJM's safe configuration snapshot and per-email observer are a strong foundation but do not replace protocol diagnostics.

Gate 9 - Publish the evidence

The title cannot be claimed from unit tests alone. Release an executable, reproducible conformance and robustness suite covering:

- Postfix, Exim, OpenSMTPD, and scripted pathological SMTP peers; representative submission services where licensing and test accounts permit.
- Every disconnect boundary: before greeting, after EHLO, during TLS, after AUTH, after MAIL, between pipelined RCPT replies, after 354, during DATA/BDAT, after final bytes, and after final success.
- Multi-recipient mixtures of 2xx, 4xx, and 5xx; multiline replies; malformed enhanced codes; slow-loris reads; short writes; server resets; and connection reuse after failures.
- SMTPUTF8, 8BITMIME, exact SIZE, complete DSN syntax, REQUIRETLS, PIPELINING response ordering, CHUNKING/BINARYMIME, and fallback behavior.
- Strict DKIM, S/MIME, and OpenPGP verification against independent implementations.
- Large-message peak memory, streaming progress, cancellation latency, pool fairness, bounded-overload behavior, and connection leak detection.
- Throughput and tail latency with and without PIPELINING, CHUNKING, and pooling across low- and high-latency links.
- Cross-platform CI on supported JDKs, with repeatable containers and published raw results.

The suite should be runnable by competitors. That converts marketing into evidence and helps improve the ecosystem.

5. Priority map: what belongs in 10.0.0

Not every desirable feature must block 10.0.0. The architectural contracts must.

Priority	Ship before 10.0 API freeze	Why
P0	Per-recipient result/reply model, enhanced codes, indeterminate outcome, retry disposition	Downstream transport, observer, queue, and framework APIs will otherwise freeze around an incomplete truth model.
P0	PIPELINING plus complete response correlation and failure-injection tests	The central open SMTP-engine gap and the basis for efficient multi-recipient submission.
P0	First-class SMTPUTF8/8BITMIME/SIZE requirements and exact finalized-content decisions	Prevents silent corruption, late server rejection, and inconsistent signing/size behavior.
P0	Complete DSN and per-message REQUIRETLS	Closes visible parity gaps with MailKit and Nodemailer through coherent APIs.
P0	End-to-end deadlines, real cancellation outcomes, bounded async queue/backpressure	Robustness cannot rely on unbounded executor demand or future-state cancellation.
P0	Public fault-injection/conformance release gates	Necessary evidence for a robustness claim and protection against protocol regressions.

Priority	Ship before 10.0 API freeze	Why
P1	CHUNKING/BDAT/BINARYMIME with streaming/progress and exact commit semantics	High-value performance/capability work, but safer to stage after PIPELINING if schedule risk threatens correctness.
P1	Redacted transcript, capability probe, phase metrics, optional observability adapters	Major production usability; can land in an early 10.x release if the core outcome hooks are already stable.
P1	Pool health/age policy, saturation metrics, optional rate limiter	Operational leadership; build on the bounded demand contract.
P2	General transport failover policy helpers	Useful only when driven by the typed retry disposition; never blind failover after an ambiguous commit.
P2	More framework adapters and durable-queue integrations	Keep core neutral; publish adapters/examples after the terminal result contract stabilizes.

If schedule forces a choice, move CHUNKING, rate limiting, and broad integration work-not the result, cancellation, or backpressure contracts. A major version is the moment to get the semantics right.

6. What not to put into this claim

Several adjacent ambitions would dilute the work:

- **POP3/IMAP:** a legitimate ecosystem expansion, but a different product and protocol problem.
- **SaaS provider APIs:** useful adapters, not evidence of SMTP robustness.
- **A durable multi-day mail queue:** this turns the library toward MTA/application infrastructure. Expose outcomes that existing queues can consume instead.
- **Automatic retry by default:** unsafe after ambiguous completion and for recipients already accepted in a partial transaction.
- **“Exactly once” delivery:** SMTP does not offer that guarantee.
- **MTA-STX, TLS-RPT, MX selection, and recipient-domain DANE:** important for direct-to-MX mail transfer, not normal authenticated submission to a configured relay. If pursued, make it an explicit separate MTA mode.
- **ARC as a core sending requirement:** valuable for intermediaries and receivers; not necessary for ordinary message submission.
- **Template-engine ownership:** composition convenience, not SMTP reliability.

7. The public claim ladder

Use claims that track evidence rather than aspiration.

Today

One of the most complete Java frameworks for composing and submitting email.

This is credible from current breadth, but should acknowledge that 10.0.0 features remain unreleased.

With the current milestone completed and verified

The most complete open-source outbound-email framework for the JVM.

This is plausible, but should be backed by a maintained JVM comparison and should not imply that the SMTP engine beats MailKit.

With the P0 gates and public evidence

The most robust open-source SMTP submission framework: protocol-complete, recipient-precise, duplicate-risk aware, and byte-preserving.

This is the strongest defensible target. The qualifiers are testable.

Avoid

The best mailing framework in the world.

It is memorable but unfalsifiable unless “best” is tied to a published methodology. If used as an internal north star, translate it externally into measurable claims.

8. A proposed product thesis for 10.0.0

The release narrative can be simple:

Simple Java Mail 10 makes SMTP outcomes explicit. It prepares one deterministic message, negotiates the server's real capabilities, submits without silently changing protected content, records every recipient decision, distinguishes rejection from uncertainty, and tells the caller when retry is safe-or when it may duplicate mail.

This thesis unifies work that might otherwise look unrelated:

- Finalized EML is the immutable artifact.
- Rehearsal prepares and validates that artifact.
- PIPELINING/CHUNKING are negotiated ways to transmit it.
- The transport-neutral result records what the server did.
- The observer publishes one terminal truth.
- Cancellation, failover, queues, and retries consume the same retry disposition.
- Diagnostics explain the path without exposing secrets.
- Conformance tests prove the promise.

That is a framework story, not a feature checklist.

9. Standards baseline

The following standards form the practical bar. Optional extensions are not universally required, but once advertised, required, or selected their semantics are correctness obligations.

Area	Primary standards	Release expectation
SMTP and submission	RFC 5321 , RFC 6409	Correct state, reply parsing, partial recipients, commit boundary, authenticated submission profile.
TLS and identity	RFC 3207 , RFC 8314 , RFC 9325, RFC 9525	Required secure submission, post-TLS EHLO, modern TLS, SAN identity validation.
Authentication	RFC 4954 , RFC 4616, RFC 7628	Pluggable SASL, PLAIN only over verified TLS, standard OAUTHBEARER and token refresh.
International and 8-bit	RFC 6152 , RFC 6531 , RFC 6532	Inspect finalized envelope/message, negotiate explicitly, transform losslessly or fail.
Results and DSN	RFC 2034 , RFC 3463, RFC 3461 , RFC 6533	Parsed enhanced replies plus RET, ENVID, NOTIFY, and ORCPT; never call DSN proof of delivery.
Performance extensions	RFC 1870 , RFC 2920 , RFC 3030	Exact SIZE, ordered PIPELINING correlation, correct DATA/BDAT/BINARYMIME paths.
Transport requirement	RFC 8689	Per-message REQUIRETLS with explicit unsupported/failure behavior.
Message and crypto	RFC 5322, RFC 2045-2049, RFC 6376 , RFC 8551 , RFC 3156 , RFC 9580	Legal MIME and headers, deterministic bytes, current DKIM/S/MIME/OpenPGP behavior and independent verification.

The ongoing 5321bis and 5322bis drafts should be tracked, but they are not yet the normative basis for a 10.0.0 claim.

10. Decision checklist

Before calling 10.0.0 the grand release, answer “yes” to these questions:

- Can the API represent every RCPT reply and the final DATA/BDAT reply without parsing exception strings?
- Can it say “may have been accepted” without collapsing that into failure?
- Does it derive retry safety without promising idempotency?
- Are SMTPUTF8, 8BITMIME, SIZE, DSN, and REQUIRETLS explicit policies tied to the finalized bytes?
- Does PIPELINING correlate every response and handle mixed recipient results?
- Are all queues bounded and all waits deadline-aware?
- Does cancellation reach the underlying protocol and preserve ambiguity honestly?
- Can users diagnose negotiated capabilities and TLS without leaking secrets or message bodies?
- Do DKIM, S/MIME, OpenPGP/MIME, rehearsal, SIZE, send, and retry consume the same immutable content?
- Can an independent developer run the conformance and fault-injection suite and reproduce the published claims?

If any of the first seven answers is no, the framework may still be excellent, but “most robust” is premature.

Conclusion

Simple Java Mail does not need to become an SMTP server, an inbox client, or a SaaS abstraction. It needs to become the framework that refuses to lie about submission.

The strategic opening is real because no surveyed competitor owns the entire path from rich message construction to exact wire content, modern capability negotiation, bounded execution, complete recipient evidence, and safe retry semantics. MailKit is the protocol target. Nodemailer is the operations target. Symfony is the integration target. SJM's opportunity is to combine those strengths around a stronger invariant:

For every message, preserve the bytes; for every recipient, preserve the reply; for every interruption, preserve the uncertainty.

Build and publicly prove that invariant, and the title becomes a reasonable conclusion rather than a slogan.

Research references

Simple Java Mail

- [Simple Java Mail repository and current release](#)
- [Stable feature overview](#)
- [Stable configuration and pools](#)
- [Feature matrix](#)
- [10.0.0 milestone](#)
- [Issue #699: PIPELINING and CHUNKING](#)
- [Issue #574: Delivery Status Notification support](#)

Libraries

- [MailKit project](#), [Smtplib API](#), [capabilities](#), and [telemetry](#)
- [MimeKit project](#)
- [Nodemailer SMTP](#), [pooling](#), [DSN](#), and [stream transport](#)
- [Symfony Mailer](#) and [Messenger](#)
- [Vert.x Mail Client](#)
- [lettre](#)
- [go-mail](#)
- [Python smtplib](#), [email](#), and [aiosmtplib](#)
- [PHPMailer](#)

Standards and registries

- [RFC Editor](#)
- [IANA SMTP service extension registry](#)
- [IANA enhanced status code registry](#)
- [SMTP 5321bis draft status](#)

Limitations. This is a source-based comparative study, not a fresh black-box benchmark of every library. Some negative findings are inferences from complete public APIs and current source rather than explicit vendor statements. Commercial proprietary SMTP SDKs were not exhaustively surveyed. Feature and release status can change after the snapshot date. The 10.0.0 branch is unreleased; stable documentation describes 9.3.2 unless an issue or local branch audit is stated.